

Workflow Failure Modes II: Unsafe Action, Drift, and Recovery

FRIDAY AI Education — Episode 031 · 2026-07-17 · Printable Companion

FRIDAY AI Education — Episode 031 · Printable Paper

Lesson 17 of 37 in the AI-Native Operator arc · Workflow Failure Modes, part 2 of 2

Abstract

The previous lesson covered failures of *belief*: workflows that go wrong because they ingest bad inputs or report conclusions with more confidence than the evidence supports. This paper covers the second, more expensive half of the failure taxonomy — failures of *action*. These are the cases where a workflow does something to the world (sends a message, moves money, schedules a job, publishes content) and the failure lives in the doing: the action was unsafe to take, the system had quietly drifted away from its assumptions before the action ran, nobody clearly owned the workflow when it broke, or there was no path back once the damage was done. We work through five concepts — drift, ownership, rollback, monitoring, and incident recovery — using three concrete cases: an unsafe outbound send, a rate-limited audio pipeline, and a scheduler that fails silently for eleven days. The organizing claim is simple: **a workflow's risk is not determined by how smart its reasoning is, but by how irreversible its actions are and how quickly its failures become visible.** By the end you should be able to look at any automated workflow and answer four diagnostic questions: What can this thing do to the world? How would I know it's degrading? Who picks up the phone when it breaks? And what does "undo" mean here — if it means anything at all.

1. Where This Sits: From Wrong Beliefs to Wrong Actions

Lesson 16 established a useful dividing line. A workflow can fail *epistemically* — it believes something false because its inputs were bad, or it presents a shaky conclusion as a solid one. Those failures are contained inside the workflow's head, so to speak. The damage happens only when a human or downstream system trusts the output.

Today's failures are different in kind, not just degree. When a workflow **acts** — when it crosses from computing things to changing things — three properties of the failure change simultaneously:

1. **The blast radius moves outside the system.** A wrong draft is a wrong draft. A wrong *sent email* is a relationship problem, a legal problem, or a reputational problem.
2. **Time starts working against you.** Epistemic failures wait patiently to be discovered. Action failures compound: a bad scheduled job runs again tomorrow, and the day after.
3. **Recovery becomes a first-class design question.** You can fix a wrong belief by correcting it. You cannot un-send, un-charge, or un-publish by editing a variable.

This is why serious operators treat "can reason well" and "can act safely" as separate engineering problems. The industry's risk frameworks agree — the NIST AI Risk Management Framework, which anchors this episode's reading, organizes its guidance around exactly this: not "is the model good?" but "is the *system's deployment* governed, mapped, measured, and managed?" More on that in Section 8.

One sentence of continuity and we move on: everything from the memory unit — retrieval, provenance, forgetting — still applies here, because acting systems consume memory as input; but today we assume the inputs were fine and ask what still goes wrong.

2. Unsafe Action: The Send Boundary

2.1 The core mechanism

Every workflow that touches the world has somewhere in it a line we'll call the **send boundary**: the point where output stops being internal state and becomes an external fact. Before the boundary, everything is a draft. After it, everything is history.

The defining property of the send boundary is **irreversibility**, and irreversibility is not binary — it's a spectrum worth internalizing:

- **Trivially reversible.** Writing a file in your own workspace. Updating a record you control. Cost of error: seconds.
- **Reversible with effort.** Merging code (revert exists, but downstream builds may have consumed it). Publishing a page (you can take it down, but caches and crawlers may have copies). Cost of error: hours, plus residue.
- **Reversible only socially.** Sending an email or Telegram message. You can send a correction, but you cannot make the recipient un-read the original. Cost of error: measured in trust, not time.
- **Irreversible.** Executing a payment to an external party. Deleting data with no backup. Disclosing confidential information. Cost of error: whatever it is, you're paying it.

The engineering discipline that falls out of this is one sentence long: **match the strength of your safeguards to the irreversibility of the action, not to the sophistication of the reasoning that produced it.** A brilliant model composing a payment instruction deserves *more* scrutiny than a dumb script renaming files, not less — because the brilliance is irrelevant to what happens if it's wrong.

2.2 Case one: the unsafe send

Concrete version. A founder builds an automation: an agent drafts replies to inbound investor and partner emails, using context pulled from notes and prior threads. For three weeks it runs in draft mode — the agent writes, the founder approves, the message goes out. The drafts are consistently good. So, reasonably enough, the founder removes the approval step for "routine" messages to save twenty minutes a day.

Week five: an inbound email arrives from a journalist, phrased like a friendly partner inquiry. The classifier tags it routine. The agent, doing exactly what it was built to do, replies helpfully — including two lines about an unannounced product direction that lived in the founder's notes. Nothing hallucinated. Nothing retrieved incorrectly. Every component worked as designed. The failure was **architectural**: an action with reputational irreversibility was gated by a classifier whose error rate had never been measured against the cost of its errors.

Three lessons compress out of this case:

First: approval gates are not a statement about model quality. The temptation to remove the gate came from the drafts being good. But the gate was never there because the drafts might be bad — it was there because *sends are irreversible* and drafts are not. Model quality improving does not change the irreversibility of the action. This is the single most common reasoning error in agent deployment, and it feels like progress while you're making it.

Second: the correct control is often structural, not intelligent. The robust fix is not "a smarter classifier for detecting journalists." It's rules that don't reason at all: no autonomous send to any address not previously corresponded with; no autonomous send containing content sourced from private strategy notes; hard caps on autonomous sends per day. Dumb rules have a property clever classifiers lack — their failure modes are enumerable. You know exactly what a whitelist won't catch. You never fully know what a classifier won't catch.

Third: the human-in-the-loop must be load-bearing, not decorative. If the approval step is a person reflexively clicking "approve" forty times a day, you have the latency of a manual process with the safety of an automated one — the worst quadrant. Good gates are *narrow* (only genuinely risky actions reach a human, so attention stays fresh) and *informative* (the approval screen shows why this one was flagged, not just the payload).

2.3 The general test

For any workflow, before granting it the ability to act, write down its **action inventory**: the complete list of world-changing operations it can perform, each tagged with its position on the reversibility spectrum. If you can't produce this list, the workflow isn't ready to act autonomously — not because it's unintelligent, but because *you* don't yet know what you're authorizing. An agent with an unenumerated action surface is not an employee with initiative; it's an unsigned blank check.

3. Drift: The Failure That Arrives on No Particular Day

3.1 The core mechanism

Drift is the gradual divergence between what a system assumes and what is actually true, in the absence of any discrete breaking event. It is the most underdiagnosed failure mode in automated workflows, for a structural reason: our debugging instincts are built around *events*. Something broke; what changed? Drift breaks that instinct because the honest answer is "nothing changed — a hundred things each changed two percent."

Drift comes in recognizable flavors:

- **Environmental drift.** The world the workflow operates on shifts: an API tightens its rate limits, a website changes layout, a partner changes their email format, a data source alters its schema. Your code is byte-identical to the day it worked perfectly.
- **Data drift.** The statistical character of the inputs shifts. The email classifier trained on last quarter's inbox meets this quarter's inbox — new senders, new topics, new tone — and its accuracy erodes without a single error being thrown.
- **Configuration drift.** Small manual interventions accumulate: a timeout bumped during an incident and never reverted, a cron entry edited by hand, a credential rotated on one machine but not another. Each change was individually sensible. Their *sum* is a system no one has a complete picture of.
- **Assumption drift.** The quiet one. The workflow still matches reality, but reality's *owners* have moved on — the process it automates was redesigned, the person it reports to changed roles, the business no longer needs the thing it faithfully produces. It runs correctly and pointlessly.

The unifying property: **drift produces degradation, not errors.** Systems that only alarm on exceptions are structurally blind to it.

3.2 Case two: the rate-limited audio pipeline

A daily pipeline turns written briefs into audio episodes: fetch the source, generate the script, call a text-to-speech API, publish the file. It has run flawlessly for two months.

The TTS provider, dealing with growth, quietly tightens rate limits on the pipeline's tier. No email, or an email that landed in a folder nobody reads. The pipeline's retry logic — well-built, exponential backoff — absorbs the new limits invisibly at first. Then the accommodations compound:

- **Week 1:** Retries succeed on the second or third attempt. Episodes publish 20 minutes later than usual. Nobody's threshold for "late" is that tight.
- **Week 2:** Under heavier throttling, some long episodes hit the retry ceiling on their final audio segments. The pipeline, built to be resilient, publishes what it has. Episodes are now occasionally truncated. They still *play*. The success log still says success.
- **Week 3:** The operator notices an episode ends mid-sentence, assumes a one-off glitch, re-runs it manually. It works (off-peak, lighter throttling), which *confirms the wrong theory*. This is worth underlining: drift often survives its first detection because a single retry succeeding looks like evidence of a transient fault rather than a trend.
- **Week 5:** A listener asks why the last several episodes have been short. Investigation finally begins. The root cause — a policy change five weeks earlier at a vendor — takes a day to find, because nobody is looking for something that happened five weeks ago. They're looking for what changed *yesterday*.

3.3 What actually catches drift

Notice what failed here: not the code, and not even the monitoring the pipeline had (it monitored *completion* — did the job finish?). What was missing is the distinction between two kinds of observation:

Liveness monitoring asks: *did it run?* Job completed, exit code zero, file published. Necessary, and completely blind to drift — the audio pipeline was "live" throughout its decay.

Health monitoring asks: *is what it produced still normal?* Episode duration versus trailing average. Retry count per run. API latency per call. Time-to-publish. These are **trend metrics**, and drift is visible in them weeks before it's visible to users — retry counts started climbing on day one of the throttling.

The practical discipline: for every workflow, define two or three **canary metrics** — cheap, continuously-collected numbers whose drift would indicate the workflow's health is degrading even while it "succeeds." Then alert on *trend deviation*, not just absence. A pipeline whose retry rate triples in a week should page someone even if every job eventually completes. The vendor's rate-limit change was never going to announce itself; the retry counter announced it immediately, to an audience of no one.

One dry observation before moving on: resilient engineering — retries, graceful degradation, publish-what-you-have — made this failure *worse*, not better, because resilience without observability converts loud failures into quiet ones. Robustness that hides its own activation is a drift accelerant. Every fallback path should increment a counter somewhere a human will eventually look.

4. Silent Failure and Ownership: The Scheduler That Told No One

4.1 Case three: the silent scheduler

A team runs a nightly job: reconcile the day's records, flag anomalies, deliver a summary to a shared channel. It has run for months. It is, in everyone's mental model, "handled."

During routine infrastructure maintenance, the machine hosting the scheduler is rebuilt. The scheduler's configuration lives in a file that wasn't in the rebuild checklist — it had been added by hand, eighteen months and one departed engineer ago. The rebuilt machine comes up healthy. Every service on the checklist passes. The nightly job simply ceases to exist.

Here is the trap in its purest form: **a job that doesn't run doesn't fail**. There is no error, no exception, no crash — the code that would have produced an alert never executes. The monitoring, such as it was, watched the job's *output* for errors. Absence of the job produced absence of errors. Green across the board.

Eleven days later someone asks in the channel, "did anything weird happen with Tuesday's numbers?" and scrolls up to find the last summary is from a week and a half ago. Eleven days of anomalies unflagged. The recovery is tedious in exactly the way silent failures always are: not just restoring the job, but reconstructing what it *would have caught* during the gap — an eleven-day backfill nobody budgeted for.

4.2 The dead man's switch

The technical fix is old, cheap, and worth knowing by name. You cannot alarm on the absence of a signal from *inside* the thing that's absent. So you invert the arrangement: the job, on every successful run, checks in with an independent watcher (a **heartbeat**). The watcher's only rule: *if the check-in doesn't arrive by its deadline, alert*. This is a **dead man's switch** — silence itself becomes the alarm condition.

The design principle beneath it generalizes: **every scheduled workflow needs a watcher that does not share the workflow's failure modes**. If the monitor runs on the same machine, in the same scheduler, behind the same config file, then the event that kills the job kills the witness too. The watcher must be boring, separate, and dumb — its entire job description is "notice silence."

If you take one implementation detail from this paper, take this: go find your most important scheduled job and ask, *if this stopped running tonight, what is the mechanism — not the hope — by which a human finds out, and how many days would it take?* If the honest answer is "someone would eventually notice," you've found your silent scheduler. Every company has at least one.

4.3 Ownership: the organizational half of the failure

The config file being off-checklist was the proximate cause. The root cause is organizational, and it has a name: **nobody owned the workflow**.

Ownership, precisely defined, means a specific person can answer four questions about a workflow without research:

1. **What does it do, and why does it still need to exist?**
2. **How do I know it's healthy right now?** (Not "it usually works" — where do I *look*?)
3. **What do I do when it breaks?** (Where's the runbook, where are the credentials, what's the restart procedure?)
4. **What depends on it, and what does it depend on?**

The scheduler failed all four. Its creator had left; knowledge left with him. No one inherited it because no *transfer of ownership* ever occurred — the job just kept running, and a running system feels owned even when it's orphaned. This is the general law: **automation without ownership doesn't remove the human from the workflow; it removes the human's attention while keeping their liability**. The work still belongs to someone — you just don't find out who until it breaks, at which point it belongs to whoever is standing closest.

The operational discipline is unglamorous: an inventory. Every automated workflow in the company, listed, with a named owner (a person, not a team — teams diffuse, people answer), a health-check location, and a runbook link. When the owner leaves or changes roles, the inventory forces an explicit handoff. The inventory itself needs an owner, yes — it's owners all the way up, but the recursion terminates quickly at "whoever runs operations," which is rather the point of having someone run operations. For a small company this is one page. The page is not bureaucracy; it's the difference between an eleven-day outage and an eleven-minute one.

5. Rollback: Designing the Way Back Before You Need It

5.1 The core mechanism

Rollback is the capability to return a system to a known-good state after a bad change or bad action. The essential insight is temporal: **rollback is a property you build in advance, not a maneuver you improvise during an incident**. If the recovery path wasn't designed before the failure, then during the failure you aren't recovering — you're doing novel engineering under the worst possible conditions, at 2 a.m., with the blast radius growing.

Rollback capability decomposes into three questions:

Do you know what "good" was? You can't return to a state you never recorded. This is why version control, config-as-code, and database snapshots aren't hygiene theater — they are, literally, the address of home. The silent scheduler's config file existed only on one machine, in one copy, which is to say: the known-good state was never known.

Can you get there mechanically? A rollback that requires remembering seven manual steps in order is a rollback that fails under incident pressure. The bar: reversion should be *more* reliable than the deployment that caused the problem, because it will be executed by a more stressed human.

Is the state actually revertible — or has the action escaped? This loops back to Section 2. Code rolls back. Configuration rolls back. *Sent messages, executed payments, and published claims do not*. For actions past the send boundary, "rollback" is the wrong word entirely; what you have instead is **compensation** — corrective actions that offset the original (the refund, the correction email, the retraction). Compensation is always more expensive and less complete than reversion, which is precisely why Section 2's gates exist: **approval gates are what you install where rollback is impossible**. The two concepts are one design rule viewed from opposite sides of the send boundary.

5.2 The founder's decision scenario

Here's where this becomes a business judgment rather than an engineering one. Suppose you're deciding how much autonomy to grant a workflow — say, letting an agent make changes to a client-facing knowledge base without per-change review. The naive framing is "how accurate is the agent?" The correct framing is a two-by-two:

- **Reversible + monitored:** grant autonomy freely. Errors are cheap and visible. This is where you *want* aggressive automation, because iteration is nearly free.
- **Reversible + unmonitored:** fix the monitoring first. Reversibility you don't know you need is reversibility you won't use in time (see: five weeks of truncated audio).
- **Irreversible + monitored:** gate with human approval. Monitoring tells you about the disaster; it doesn't prevent it.
- **Irreversible + unmonitored:** do not automate this. Not "automate carefully" — *do not*. This quadrant is where companies acquire their formative scar tissue.

Most automation decisions that feel hard become easy once placed on this grid. The hard part is honesty about which quadrant you're in — teams routinely misfile "reversible with heroic effort" as "reversible."

6. Incident Recovery: What Happens After Something Happens

6.1 The four phases

Everything above reduces failure frequency. None of it reduces failure frequency to zero, so mature operators invest in the response as deliberately as the prevention. **Incident recovery** has four phases, and each of this paper's cases stumbled at a different one:

Detection. How long between the failure occurring and a human knowing? The scheduler's answer was eleven days; the audio pipeline's, five weeks. This gap — **time-to-detection** — is the single most improvable number in most young companies, and every section of this paper (canary metrics, trend alerts, dead man's switches) is ultimately in its service.

Containment. Stop the bleeding before diagnosing the wound. Pause the pipeline, disable the send capability, freeze the scheduler. Containment requires that a **kill switch** exists and that the responder knows where it is — another thing designed in advance or not at all. Note the ordering discipline: containment *precedes* diagnosis. The instinct to understand before acting is exactly backwards during an active incident.

Diagnosis and repair. Root-cause the failure, fix it, and verify the fix — where "verify" means observing the workflow behave correctly on real work, not merely observing the absence of the old symptom. (Recall the audio pipeline's week-3 trap: a successful manual re-run *was* the absence of the symptom, and it bought two more weeks of decay.)

Learning. The blameless post-incident review: what happened, what was the timeline, why did detection take as long as it did, and — the only question that compounds — *what class of failure does this belong to, and what single change makes that whole class less likely or more visible?* The scheduler incident, reviewed well, doesn't produce "add the config file to the checklist." It produces "any scheduled job without a heartbeat is an incident waiting to be scheduled" — a rule that fixes jobs that haven't failed yet.

6.2 Why blameless is a technical requirement, not a kindness

One organizational note, because founders set this culture in the first ten incidents whether they mean to or not. If incident reviews assign blame, people delay reporting, and delayed reporting extends time-to-detection — the exact metric you're trying to crush. Blamelessness isn't generosity; it's instrumentation. You are paying for fast, honest signal from the humans in the loop, and the currency is safety from punishment. Systems with punished messengers develop silent failures at the *human* layer, which are strictly worse than the mechanical kind because no dead man's switch catches them.

7. The Failure Modes, Side by Side

Failure mode	Signature	Why standard alerting misses it	Primary defense	Recovery path
Unsafe action	Correct-looking action with irreversible consequences	Nothing errored — the system did what it was told	Action inventory; approval gates scaled to irreversibility; dumb structural rules	Compensation, not rollback (correction, refund, retraction)

Failure mode	Signature	Why standard alerting misses it	Primary defense	Recovery path
Drift	Gradual degradation with no breaking event	Produces worse results, not errors; resilience machinery hides it	Canary metrics; trend-based alerts; counting fallback activations	Identify divergence point; re-baseline; backfill degraded output
Silent failure	Absence of activity, absence of errors	Dead code raises no exceptions; green dashboards measure the wrong thing	Dead man's switch / heartbeat with an independent watcher	Restore job; backfill the gap; audit what went uncaught
Orphaned ownership	Nobody can answer "how do I know it's healthy?"	It's organizational — no dashboard measures it	Workflow inventory with named owners and runbooks; explicit handoffs	Emergency knowledge reconstruction (expensive); then assign ownership
Missing rollback	Incident response becomes improvisation	Only visible during an incident — the worst time	Known-good state capture; mechanical reversion; the reversibility 2x2	If truly irreversible: containment + compensation + the post-incident review you should have had earlier

The table rewards a second read with one question in mind: notice that the "primary defense" column contains almost no intelligence. Heartbeats, whitelists, counters, inventories, snapshots. The defenses against action failures are overwhelmingly *boring*, and that is not a coincidence — boring mechanisms have enumerable failure modes, and Section 2 already told you why that property is precious.

8. The Governance Frame: NIST AI RMF in One Honest Page

The NIST AI Risk Management Framework is the reading anchor for this episode, and it's worth situating honestly: it is a *management* framework, not an engineering manual. It won't tell you how to build a heartbeat. What it does — and does well — is give you a defensible structure for the questions this paper has been asking ad hoc. Its four functions map onto today's material almost exactly:

- **Govern** — establish accountability structures for AI systems. This is Section 4's ownership discipline, formalized: named owners, defined roles, escalation paths.
- **Map** — understand the context and characterize what the system can affect. This is Section 2's action inventory and the reversibility spectrum: know what the thing can *do* before deciding how to control it.
- **Measure** — assess and track the system's risks and performance over time. This is Section 3's canary metrics and trend monitoring: drift is invisible to anyone who isn't measuring longitudinally.
- **Manage** — prioritize and respond to risks as they materialize. This is Section 6's incident lifecycle: detection, containment, repair, learning.

Two reasons this matters to a founder beyond intellectual tidiness. First, when you eventually sell AI-operated services into serious organizations — banks, enterprises, anyone with a risk officer — *their* diligence questions will be shaped by frameworks like this one, and answering in the framework's own vocabulary ("here is our action inventory, here are our approval gates, here is our incident process") is the difference between reading as a professional operation and reading as a demo. Second, the framework encodes a stance this whole paper shares: AI risk lives in the *deployment*, not the model. A mediocre model behind good gates, good monitoring, and clear ownership is a safer system than a frontier model wired directly to the send button.

9. Vocabulary

Send boundary. The point in a workflow where output stops being internal, revisable state and becomes an external fact. Everything before it is a draft; everything after it is history.

Irreversibility spectrum. The gradient of actions from trivially undoable (local file write) through socially reversible (sent message) to truly irreversible (executed payment, disclosed secret). Safeguard strength should track position on this spectrum.

Action inventory. The enumerated list of world-changing operations a workflow can perform, each tagged with its reversibility. The precondition for granting autonomy.

Drift. Gradual divergence between a system's assumptions and reality, absent any discrete breaking event. Produces degradation rather than errors, which is why exception-based alerting misses it.

Canary metric. A cheap, continuously collected measurement (retry count, output duration, latency) whose trend reveals degradation while the workflow still nominally "succeeds."

Liveness vs. health monitoring. Liveness asks "did it run?"; health asks "is what it produced still normal?" Drift is invisible to the first and obvious to the second.

Heartbeat / dead man's switch. An arrangement where a job checks in with an independent watcher on every successful run, and the watcher alarms on *silence*. The only reliable defense against jobs that fail by not existing.

Ownership. The state in which a specific named person can answer, without research: what a workflow does, how to check its health, what to do when it breaks, and what depends on it.

Rollback. The pre-built capability to return a system to a recorded known-good state. Distinct from **compensation**, which is what you're left with past the send boundary: corrective actions (refunds, retractions) that offset but never fully erase the original.

Kill switch. A pre-positioned mechanism to immediately halt a workflow's ability to act. Exists so containment can precede diagnosis.

Time-to-detection. The interval between a failure occurring and a human knowing. Usually the most improvable reliability number in a young company.

Incident recovery. The four-phase discipline of detection, containment, diagnosis/repair, and learning — with blameless review as the mechanism that makes the learning phase, and honest detection, possible.

10. Reading Guide

NIST AI Risk Management Framework — <https://www.nist.gov/itl/ai-risk-management-framework>

How to read it without drowning: the framework proper (AI RMF 1.0) is a policy document, and reading it cover to cover is neither necessary nor recommended for this lesson. Do this instead:

1. **Start with the "AI RMF Core" section** — the Govern / Map / Measure / Manage functions. Read the one-paragraph description of each function and hold Section 8's mapping in mind: you have now seen all four

functions in miniature in this paper's three case studies. Budget: twenty minutes.

2. **Skim the "characteristics of trustworthy AI" list** (valid, reliable, safe, accountable, transparent, etc.). Don't memorize it; instead, for each characteristic, ask which of today's three cases violated it. Most map cleanly — the silent scheduler is an accountability failure, the drifting pipeline a reliability failure, the unsafe send a safety-and-transparency failure. This exercise turns a compliance list into a diagnostic checklist, which is the only form in which it's useful to an operator.
3. **Skip, for now**, the Playbook and the profiles. They matter when you're implementing governance at organizational scale; today they're swamp.

The right posture toward this document: it's a vocabulary and a structure, not a revelation. Its value to you is that serious counterparties speak it.

11. Walking Questions

Carry these on the next walk; they're ordered from recall to application.

1. **Explain the send boundary in your own words**, and articulate why "the model got smarter" is never, by itself, a valid argument for removing an approval gate.
2. **Which of your current automated workflows could fail silently tonight** — and for each, what is the *mechanism* (not the hope) by which you'd find out, and what's the honest time-to-detection?
3. The audio pipeline case argued that **resilience without observability makes drift worse**. Where in your own systems does graceful degradation currently hide its own activation? What one counter would expose it?
4. Take one bounded workflow you actually run — the daily content pipeline is a fine candidate — and place each of its actions on the **reversibility 2×2** from Section 5.2. Does its current level of autonomy match its quadrant? If not, which direction is the mismatch — and note that "too gated" is a real cost too, just a quieter one.
5. **Who owns each of your workflows, by name?** If the answer for any of them is "well, me, implicitly, I suppose" — what would a real handoff document need to contain for someone else to hold the pager?

12. Looking Ahead

The next lesson — **Writing the Workflow Spec I: Objective, Inputs, and Output Contract** — is where this unit's diagnostic work turns constructive. Two lessons of failure modes have effectively been requirements-gathering: a workflow spec worth writing must declare its inputs and their validity conditions (lesson 16), its action inventory and reversibility (this lesson), its canary metrics, its owner, and its recovery path. You now know *why* each of those clauses belongs in the contract. Next, we write one.

One-Sentence Takeaway

A workflow's risk is set by the irreversibility of its actions and the visibility of its failures — so before granting any system autonomy, enumerate what it can do to the world, instrument how you'd know it's degrading, name who owns it, and build the way back before you need it.